

Git Workshop

Oct 05, 2025

ReDI School Of Digital Integration - Aarhus



Agenda

10:00 - 10:25: Setup And Introduction

10:25 - 11:00: Core Concepts & First Steps

11:00 - 11:30: Exploring History

11:30 - 12:15: Branching & Merging

12:15 - 12:45: Lunch

12:45 - 13:30: Going Back To A Previous Commit In Git

13:30 - 13:45: Resolving Conflicts

13:45 - 14:35: Remote Collaboration

14:35 - 15:00: Best Practices, Advanced Tips, Wrap-Up & Extras

Workshop goals

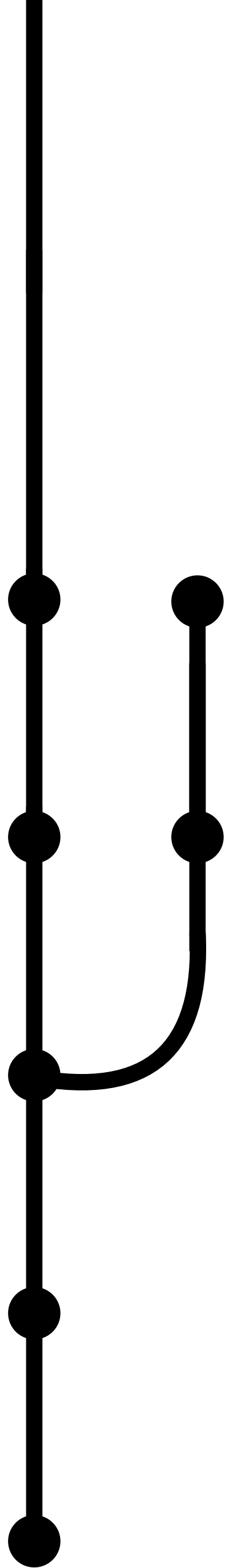
Understand Git fundamentals

Gain hands-on experience

Learn collaboration workflows

Be collaborative

Please ask questions



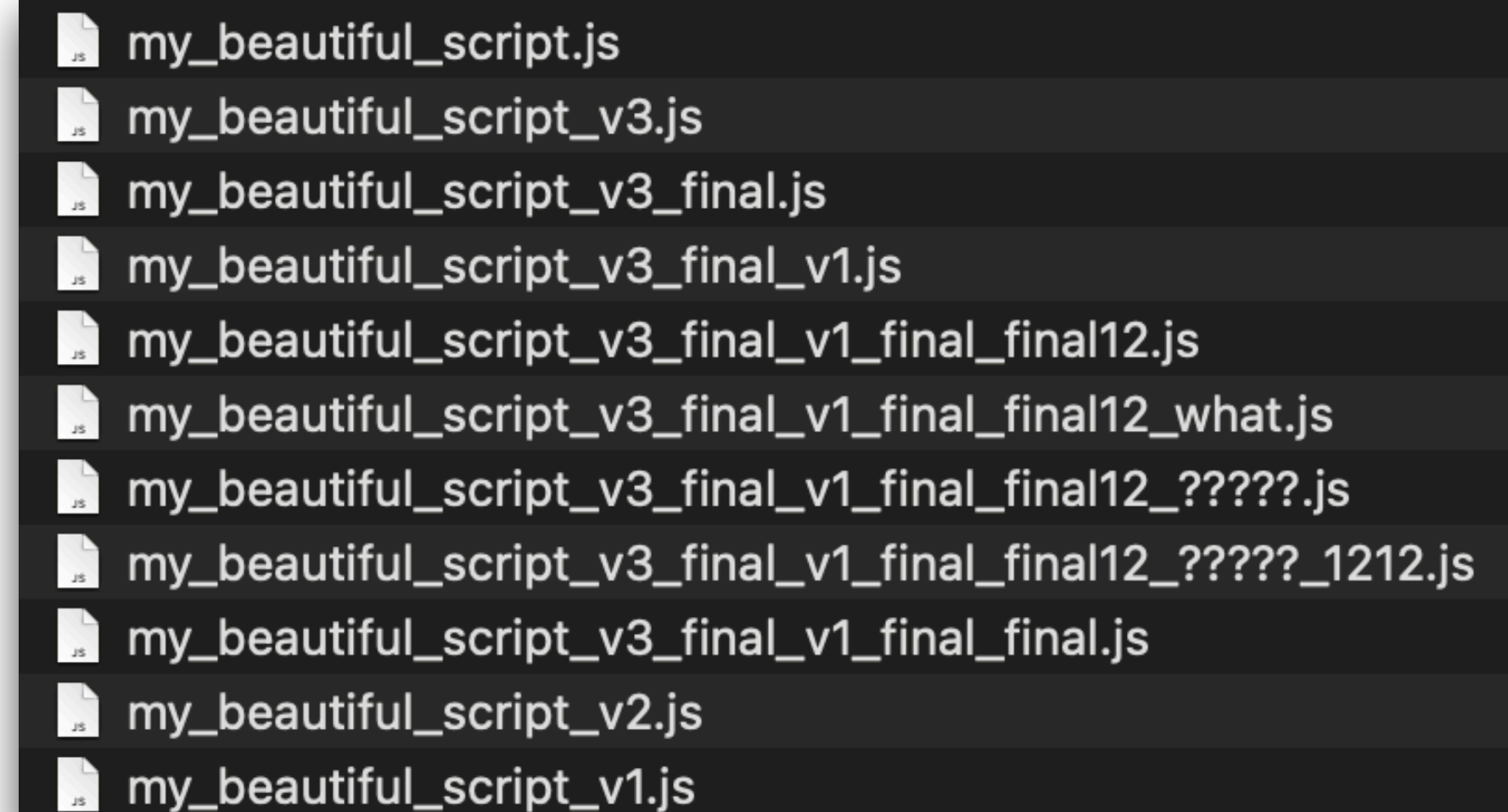
Setup & Introduction

The Problem Without Version Control

Hard to track changes.

Easy to overwrite each other's work.

Rolling back is painful



- my_beautiful_script.js
- my_beautiful_script_v3.js
- my_beautiful_script_v3_final.js
- my_beautiful_script_v3_final_v1.js
- my_beautiful_script_v3_final_v1_final_final12.js
- my_beautiful_script_v3_final_v1_final_final12_what.js
- my_beautiful_script_v3_final_v1_final_final12_?????.js
- my_beautiful_script_v3_final_v1_final_final12_?????_1212.js
- my_beautiful_script_v3_final_v1_final_final.js
- my_beautiful_script_v2.js
- my_beautiful_script_v1.js

What Is Git?

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Tracks Snapshots of files

Distributed: every clone is a full
repo

Branching and merging

Core Git Concepts at High Level

Repository (repo)

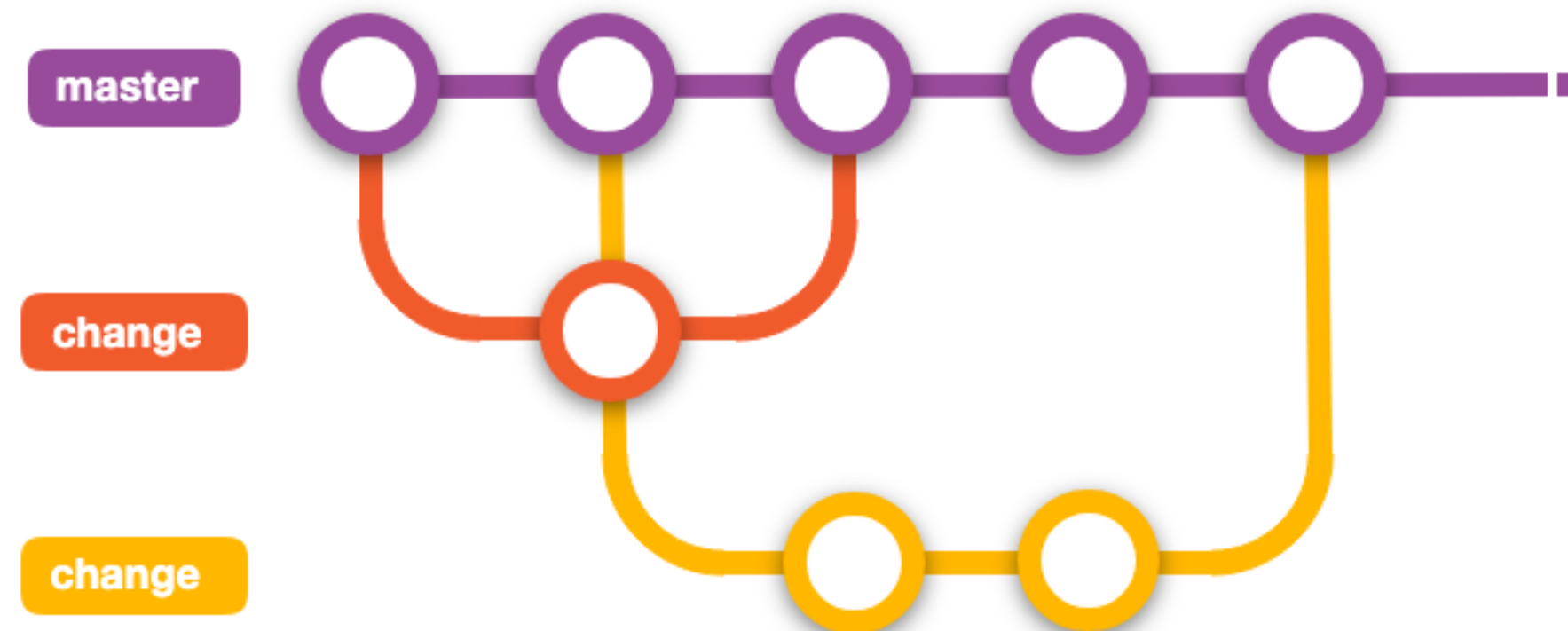
folder tracked by Git.

```
→ ~ ls -a  
Myfolder/.git
```

Core Git Concepts at High Level

Branch

a line of development.



Core Git Concepts at High Level

Commit

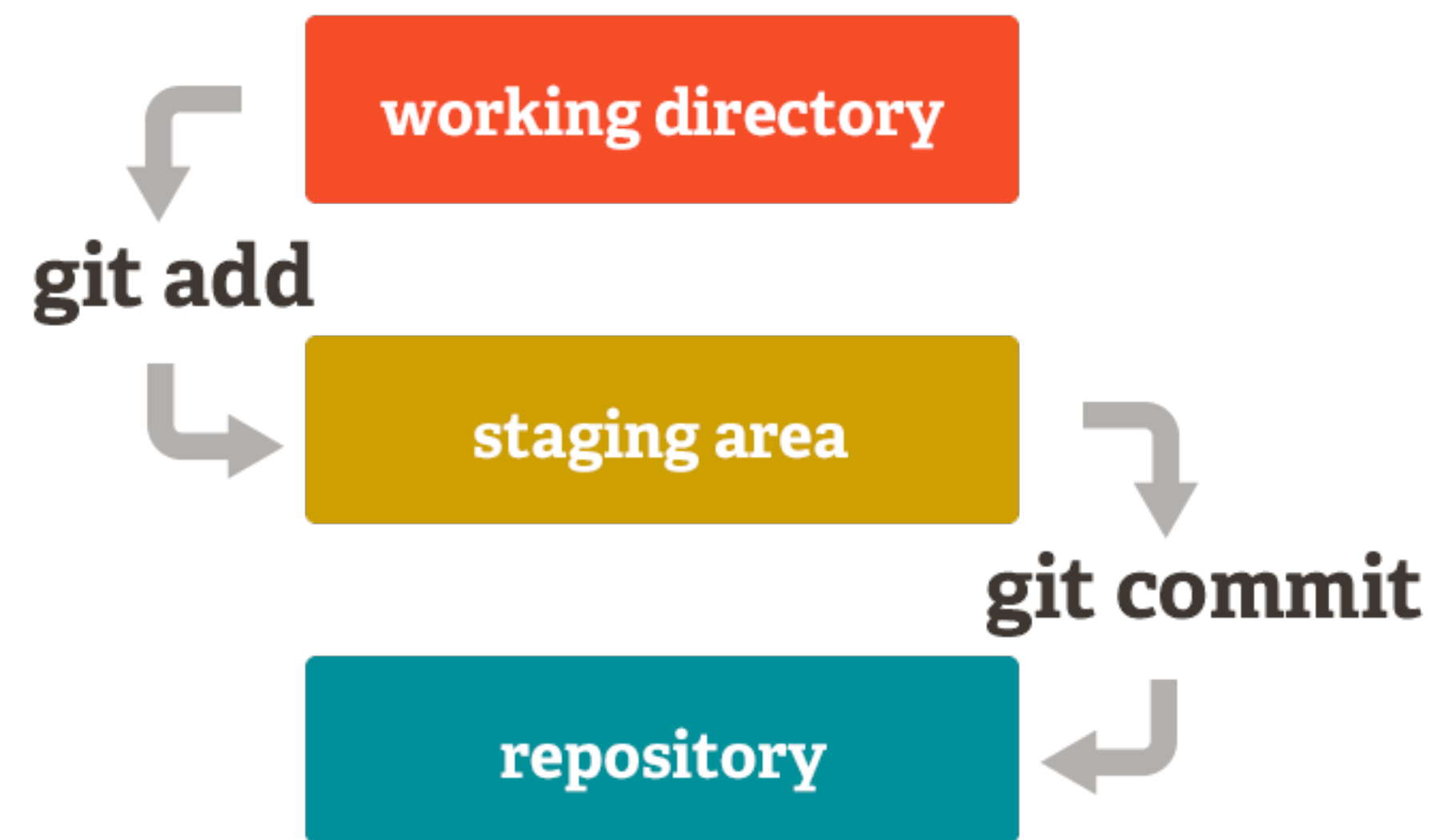
snapshot of changes, with message & author.

```
commit 30c6349bcfc31c10b6ca3fa88a4312a67c92e355
Author: Name Surname <email@gmail.com>
Date:   Tue Jun 10 21:24:01 2025 +0200
Initial commit
```

Core Git Concepts at High Level

Staging

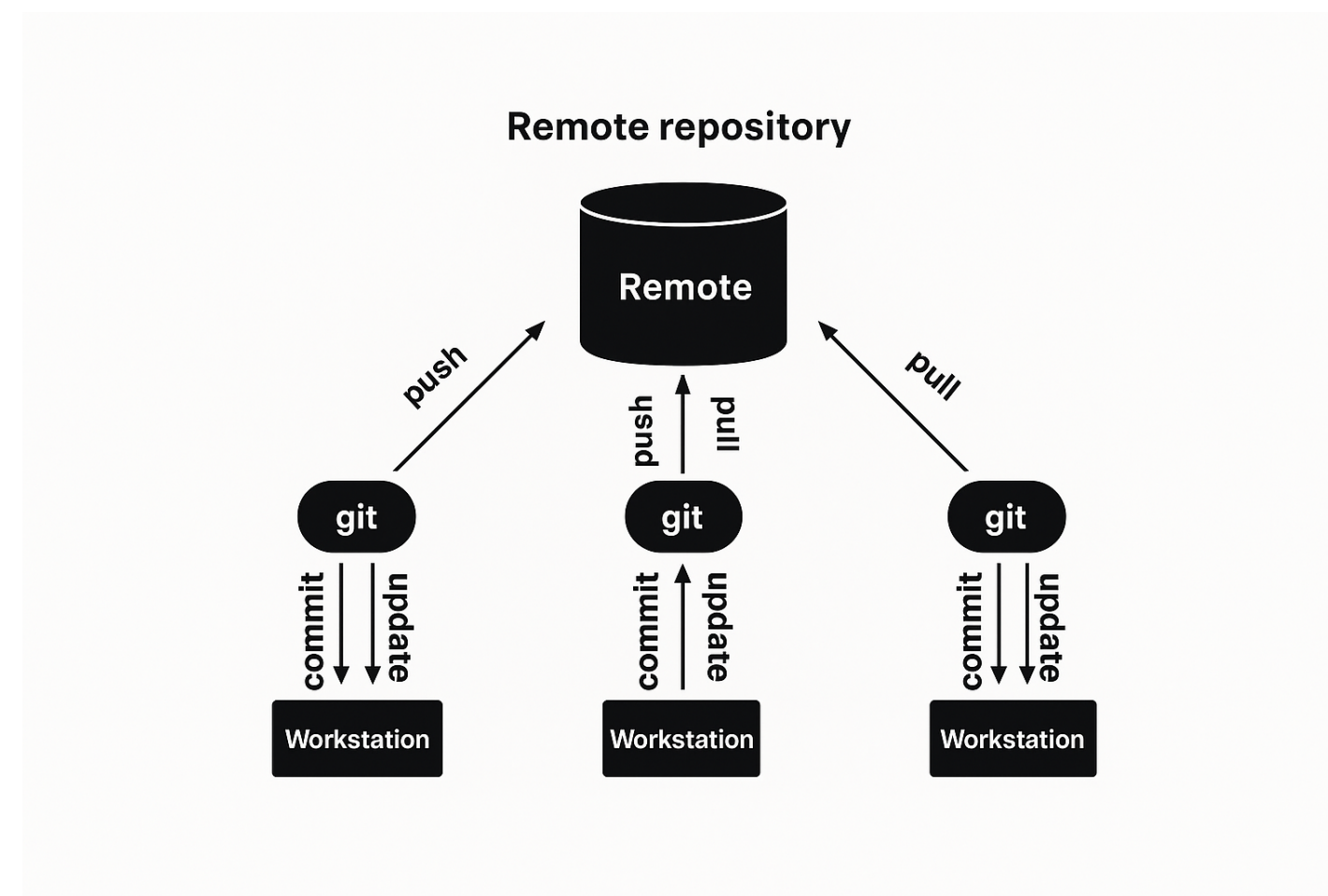
The staging area is where you prepare your changes for a commit.



Core Git Concepts at High Level

Remote

shared repo (e.g., GitHub, GitLab).



Setup Instructions

1 VERIFY

```
→ ~ git --version  
git version x.xx.x
```

2 INSTALL IF MISSING

Visit git-scm.com/downloads

During the installation process,
leave everything as default

3 CONFIGURE IDENTITY

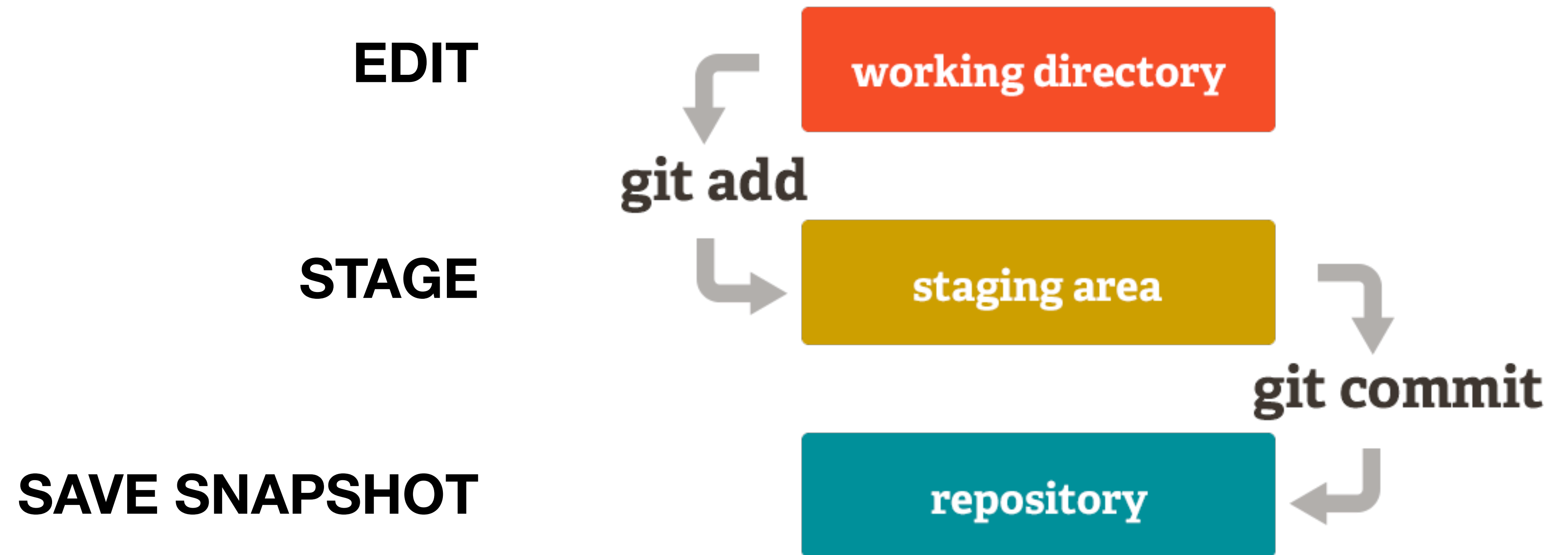
```
→ ~ git config --global user.name "Your Name"  
→ ~ git config --global user.email "Your@email.com"
```

4 CHECK CONFIGURATION

```
→ ~ git config --list  
...  
user.name="Your name"  
user.email="Your@email.com"  
...
```

Core Concepts & First Steps

The Git Workflow



Creating a Repository

1 CREATE A NEW FOLDER AND CHANGE DIRECTORY

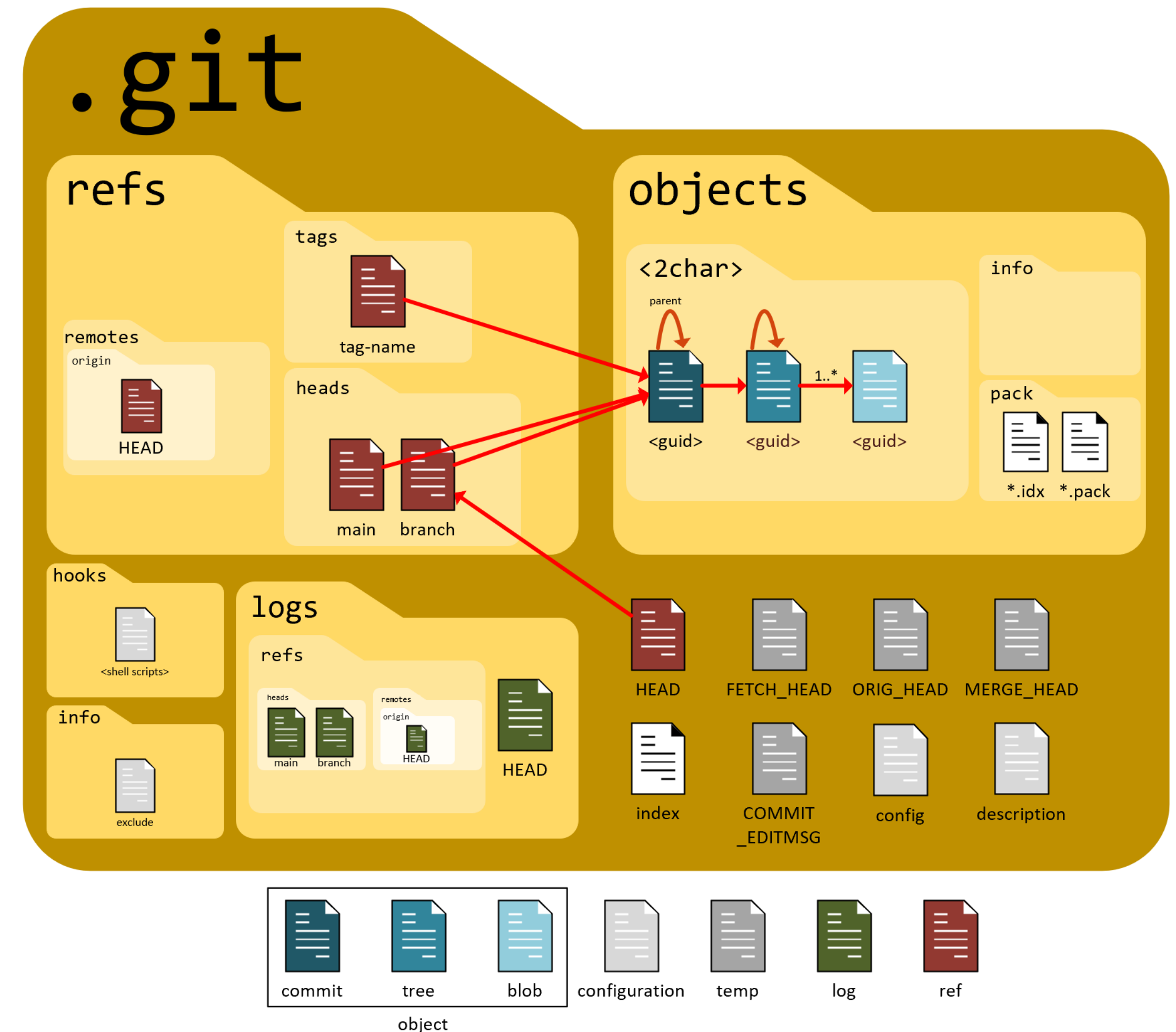
```
→ ~ mkdir my-first-repo  
→ ~ cd my-first-repo
```

2 INITIALIZE GIT FOLDER

```
→ ~ git init
```

3 CHECK THE GIT FOLDER

```
→ ~ ls -a (MACOS)  
→ ~ dir/a (WINDOWS)
```



Adding and Staging Changes

1 OPEN THE FOLDER WITH VSCODE

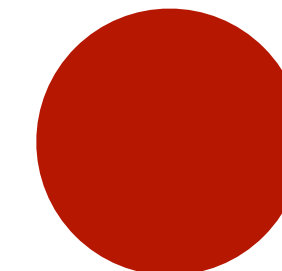
2 CREATE A NEW FILE "HELLO.TXT"

3 STAGE IT

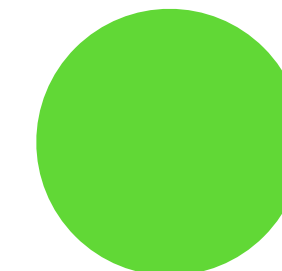
```
→ ~ git add hello.txt
```

Checking Git Status

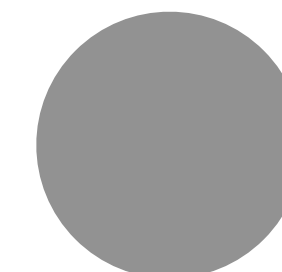
```
→ ~ git status
```



CHANGES NOT STAGED



STAGED



**"NOTHING TO COMMIT"
= CLEAN WORKING DIRECTORY.**

Committing Changes

```
→ ~ git commit -m "Add hello.txt"
```

What happens

Snapshot saved.

Commit message recorded.

Author info from git config

First Commit Complete!

You've created your first Git repo!

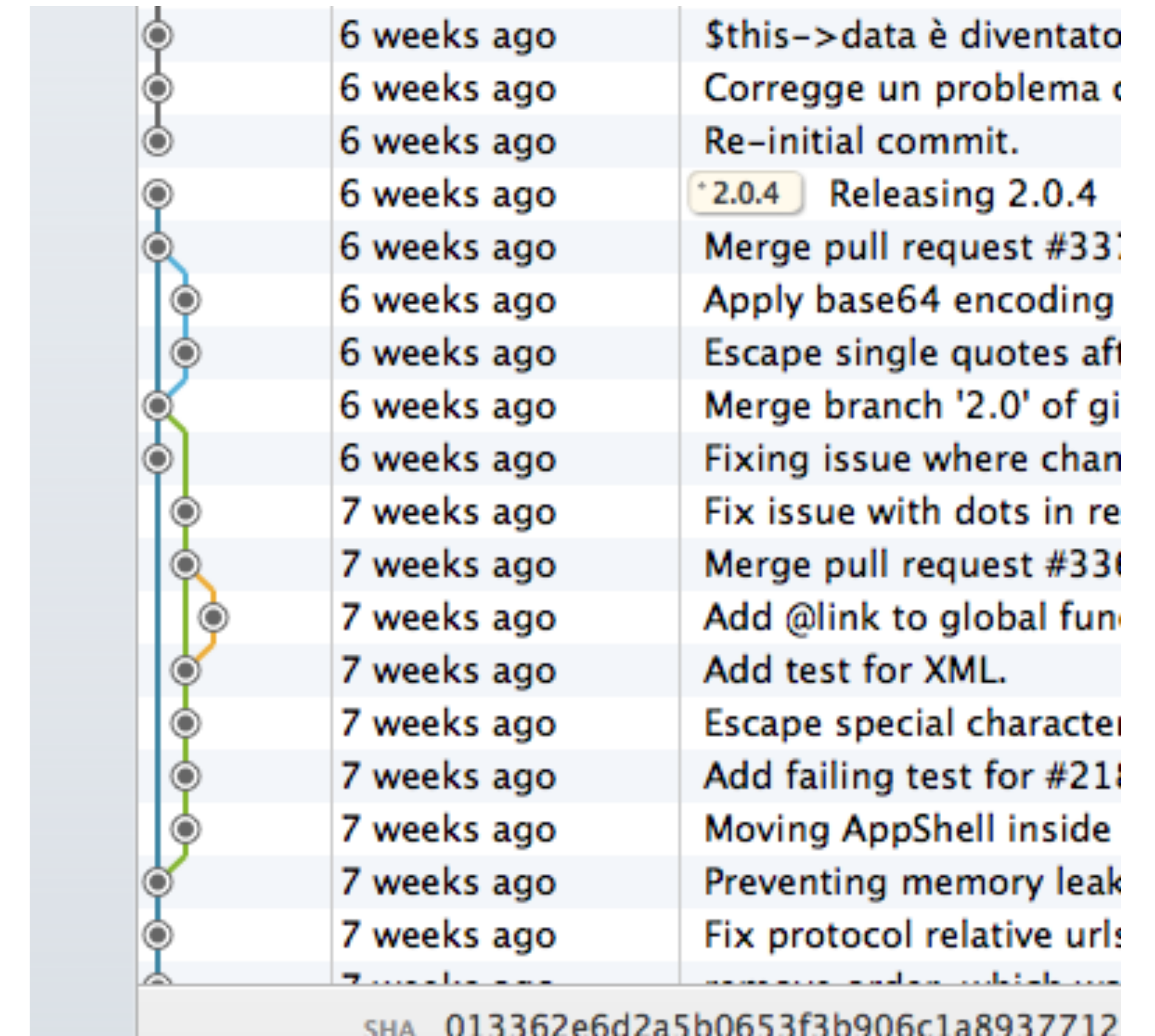
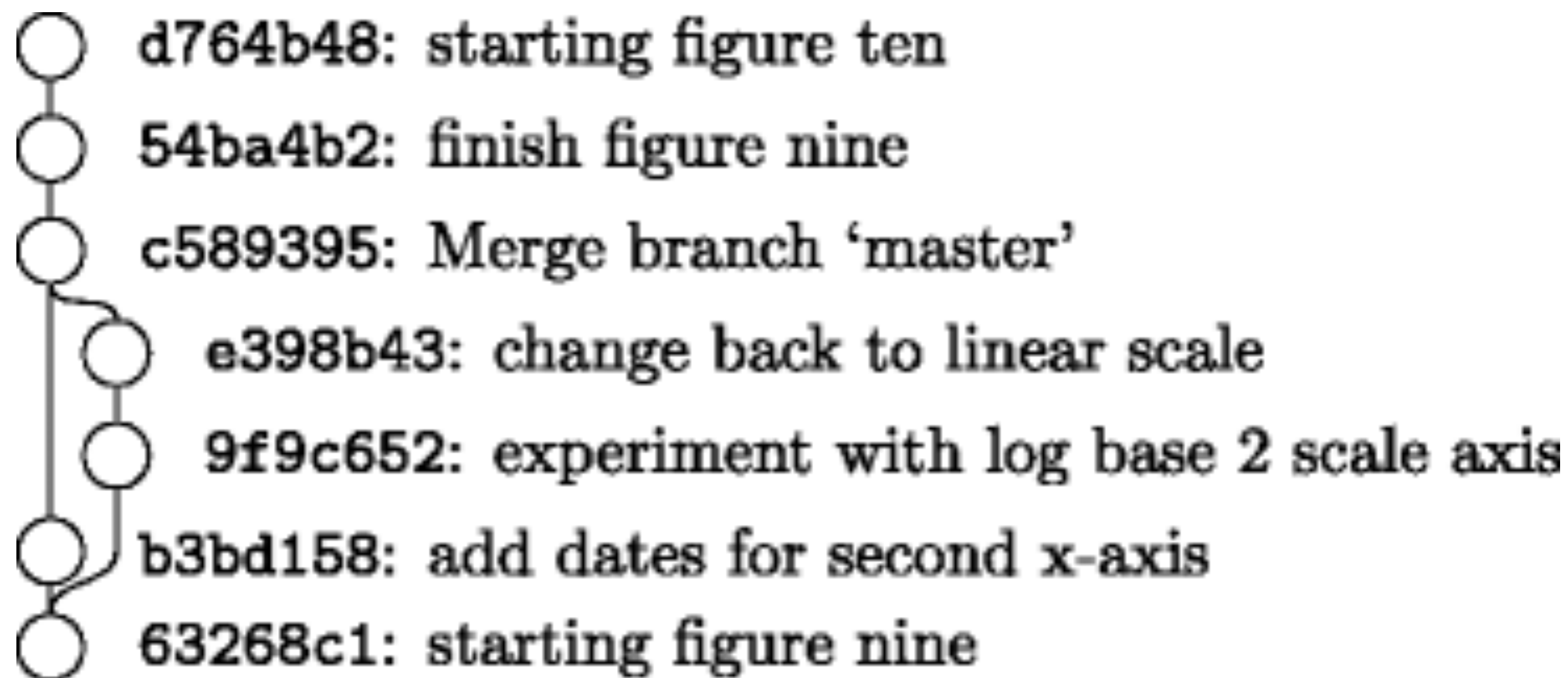
```
→ ~ git init > Start repo
```

```
→ ~ git add > Stage changes
```

```
→ ~ git commit > Snapshot saved
```

Exploring History

What is Git History?



Commit Identifiers

SHA-1 hash (first 7 chars).

8d1922ecba76596c15f583abb4896110139420ef

Viewing the Log (Basics)

```
→ ~ git log > see commit history
```

```
commit 30c6349bcfc31c10b6ca3fa88a4312a67c92e355  
Author: Name Surname <email@gmail.com>  
Date:   Tue Jun 10 21:24:01 2025 +0200  
Initial commit
```

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Log with Pretty Formats

```
→ ~ git log --graph --decorate --all  
→ ~ git log --oneline
```

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Searching in Log

```
→ ~ git log --author="name"  
→ ~ git log --grep="keyword"  
→ ~ git log path/to/file
```

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Comparing Commits

```
→ ~ git diff <commit1> <commit2>  
→ ~ git show <commit>
```

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Blame / Annotate

```
→ ~ git blame path/to/file > line by line annotated  
commits
```

Practical Use Cases

Finding when a bug was introduced.

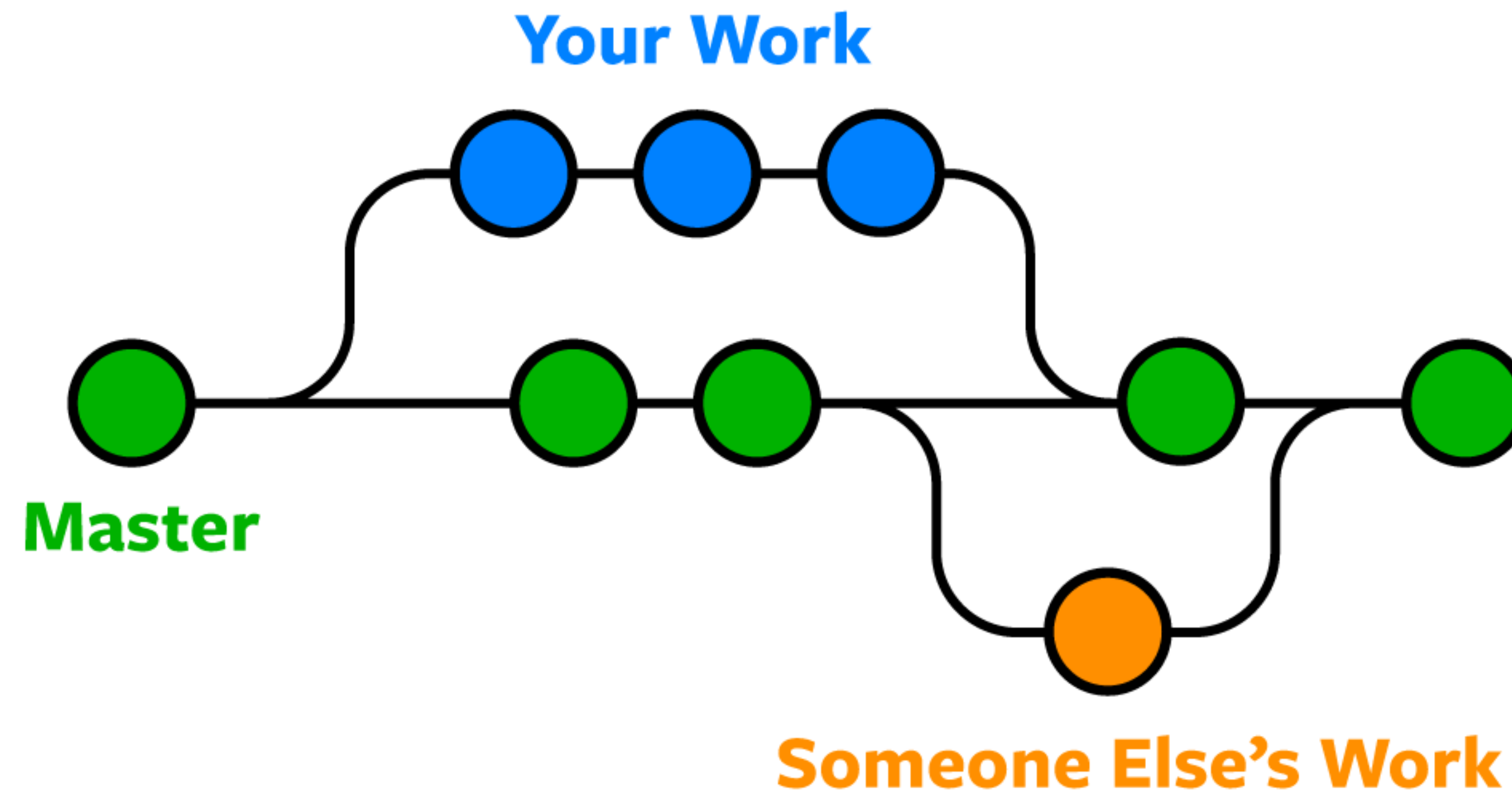
Tracking changes across team members.

Auditing for code reviews.

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Branching & Merging

What is a Branch?



Team collaboration: work in parallel

Isolation of features/experiments

Creating a Branch

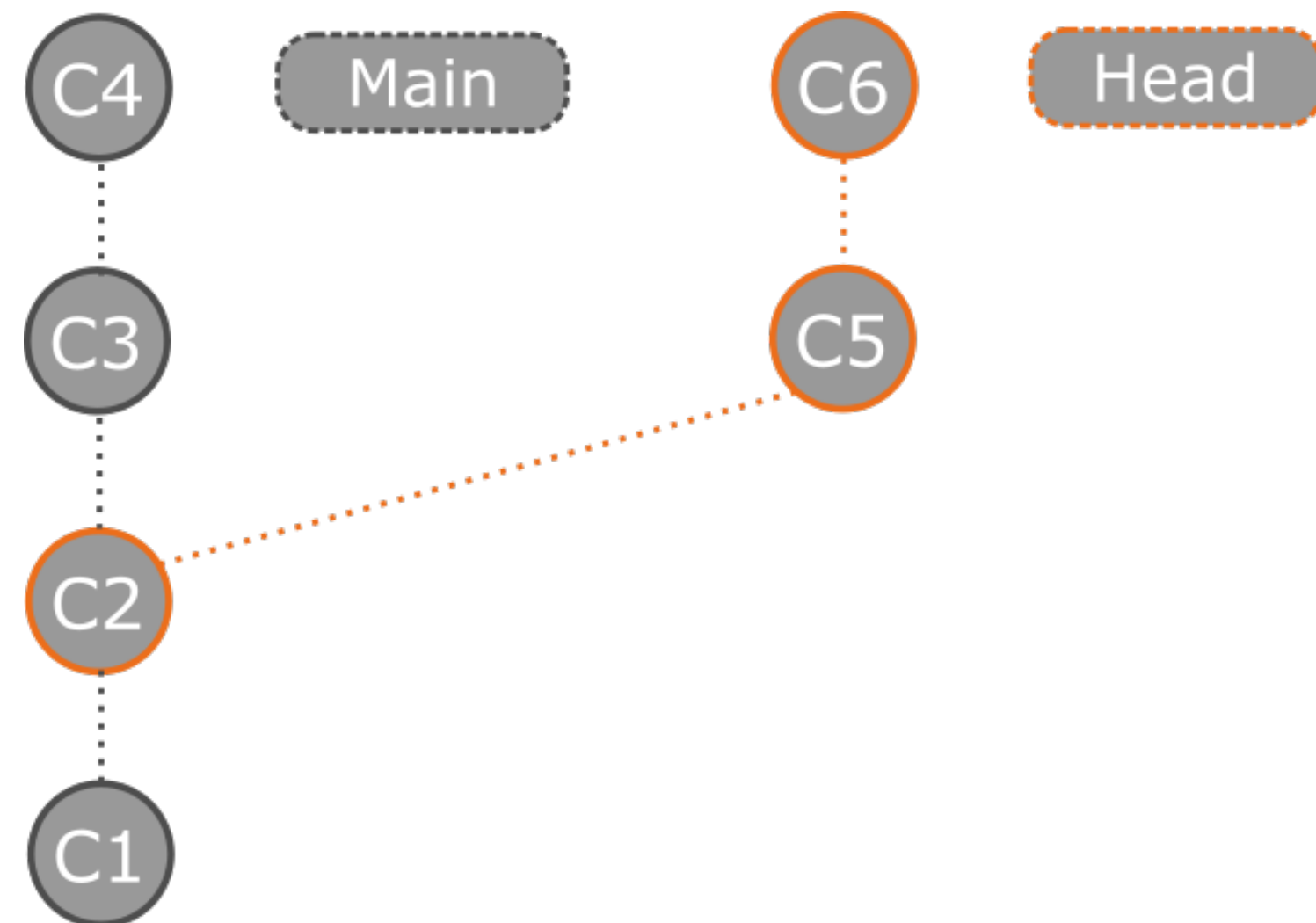
```
→ ~ git branch [feature-x]
```

Switching Branches

```
→ ~ git checkout [feature-x]
```

Creating & Switching in One Step

```
→ ~ git checkout -b [feature-x]
```



The HEAD is a pointer (a reference) to the current branch and commit you are working on.

Merging Branches

Fast-Forward Merge (no commit message)

```
→ ~ git merge [feature-x] > checkout to the base  
branch, merge feature-x into the main branch
```



Merging Branches

Non Fast-Forward Merge (commit message)

```
→ ~ git merge [feature-x] > checkout to the base  
branch, merge feature-x into the main branch
```

main ---- A ---- B ---- E
 \
 C ---- D (feature-x)



main ---- A ---- B ---- E ---- M
 \< /
 C ---- D--

Press :WQ to exit the
git editor

(M = merge commit, HEAD on main)

Visualize Branches

```
→ ~ git log --graph --decorate --all
```

When you're done viewing the log, press “q” in the terminal to quit and return to the command prompt.

Good Branching Practices

KEEP
BRANCHES
SMALL AND
FOCUSED

SYNC WITH
MAIN
REGULARLY

KEEP MAIN (OR
MASTER)
ALWAYS
STABLE

BRANCH
EARLY, MERGE
OFTEN

DELETE
MERGED
BRANCHES

Going Back to a Previous Commit in Git

Method 1

```
→ ~ git checkout <commit>
```

WHAT IT DOES

Moves your working directory to a specific commit without changing the

USE CASES

Exploring an old commit, testing code at that point.

NOTES

you are in a detached HEAD state, so new commits here won't belong to a branch unless you create one.

To save changes, run: `git switch -c <new-branch>` or `git checkout -b <new-branch>`

Method 2

```
→ ~ git reset <commit>
```

WHAT IT DOES

Moves the current branch to a previous commit, optionally changing the working directory.

USE CASES

Reset the changes to a specific coming and having the staged changes

Usually safe on local branches, not recommended on shared branches.

NOTES

—mixed
Moves branch pointer, unstaged

—hard
Moves branch pointer and resets working directory to match commit (destructive!)

Method 3

```
→ ~ git revert <commit>
```

WHAT IT DOES

Creates a new commit that undoes the changes from a previous commit.

USE CASES

Undo a commit on a branch that has been pushed/shared with others.

NOTES

Does not rewrite history; safer than reset on public branches.

Revert (the Revert)

```
→ ~ git reflog
```

WHAT IT DOES

Tracks movement of HEAD and branch tips.

USE CASES

Recover commits lost after a reset or checkout.

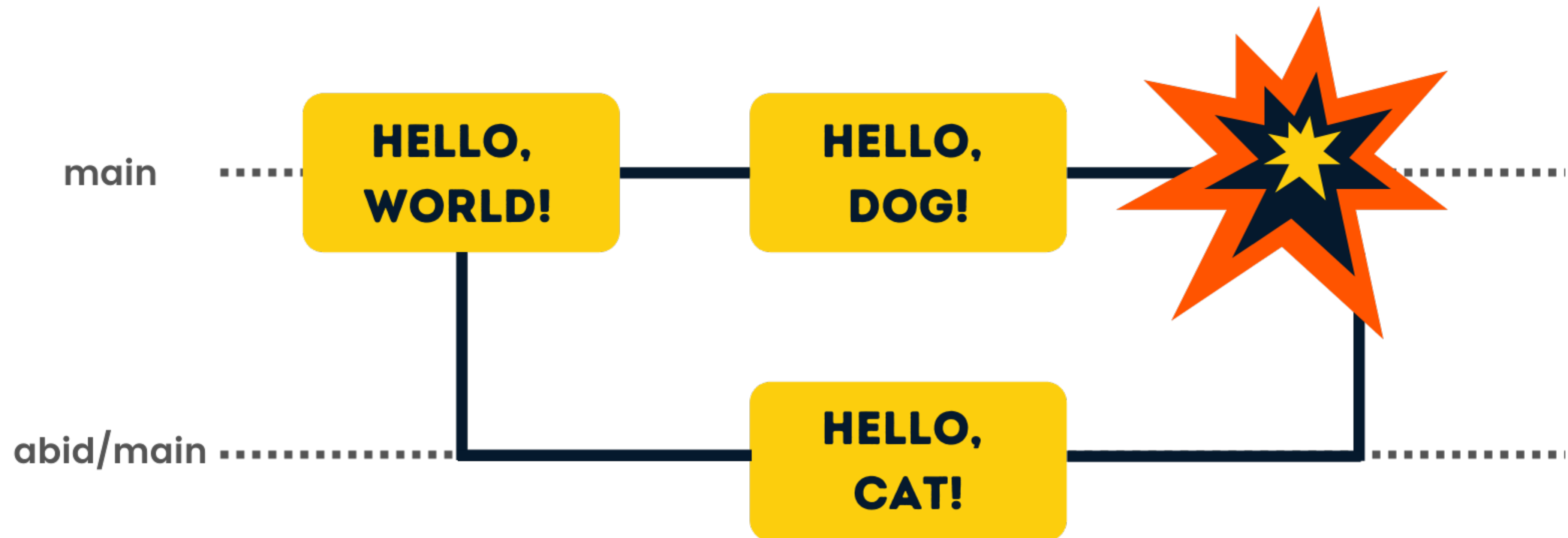
NOTES

Every change to HEAD is logged; very useful for undoing mistakes.

Resolving conflicts

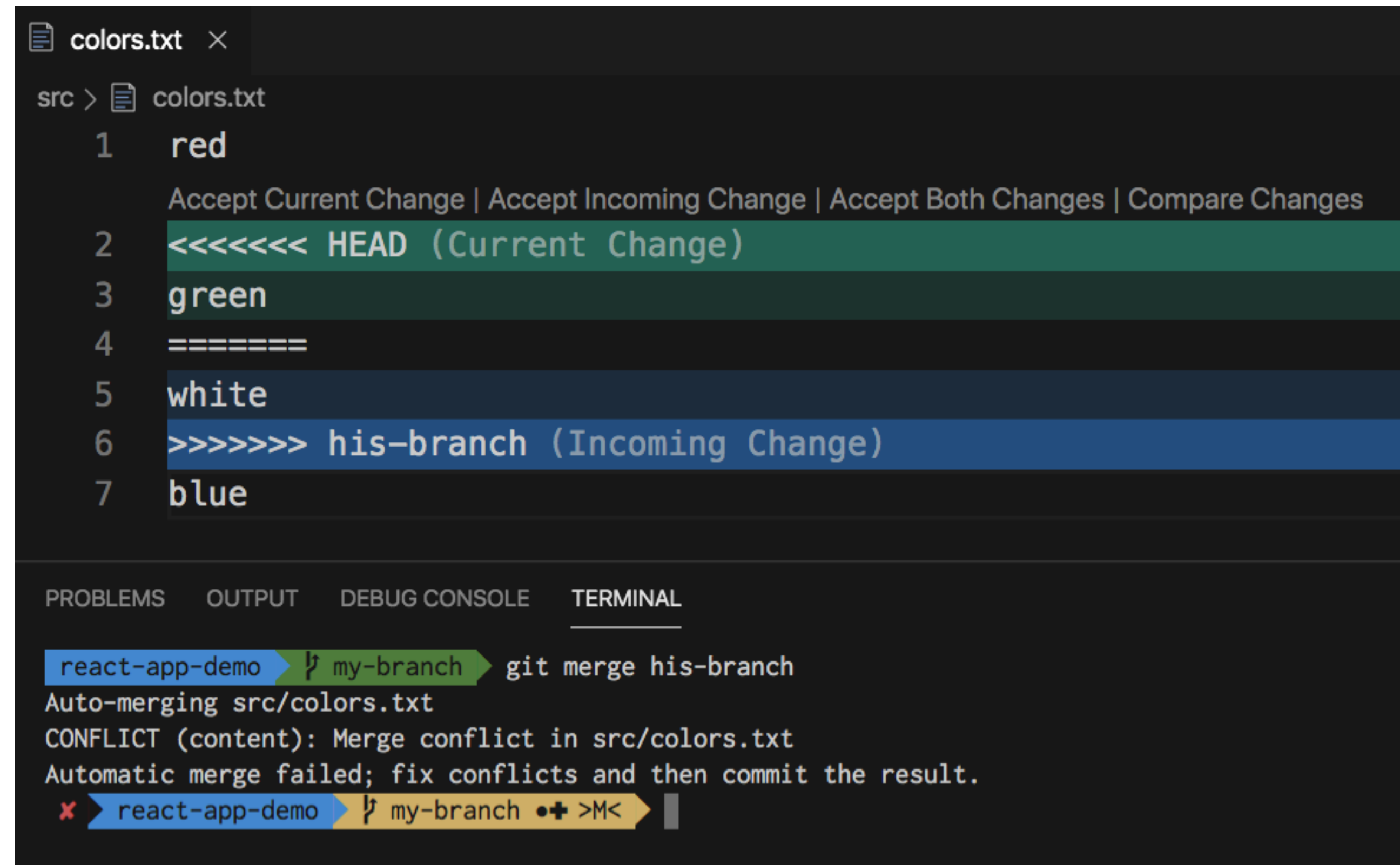
What is a Conflict?

Occurs when Git can't automatically merge changes.



Conflict Markers in Code

<<<<<<< HEAD, =====, >>>>>>> branch-name markers.



```
src > colors.txt
1  red
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
2  <<<<<<< HEAD (Current Change)
3  green
4  =====
5  white
6  >>>>>>> his-branch (Incoming Change)
7  blue

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
react-app-demo my-branch git merge his-branch
Auto-merging src/colors.txt
CONFLICT (content): Merge conflict in src/colors.txt
Automatic merge failed; fix conflicts and then commit the result.
x react-app-demo my-branch >M<
```

Steps to Resolve...

1 OPEN FILE, DECIDE WHAT TO KEEP/REMOVE.

2 STAGE THE FILE WITH GIT ADD.

3 COMMIT TO FINALIZE RESOLUTION

...however

IDE SUPPORT (VS CODE, JETBRAINS, ETC.).

OR

```
→ ~ git mergetools
```

Remote Collaboration

Setup

1 CREATE GITHUB ACCOUNT

Visit github.com

Click on the “Sign up button”

2 SIGN IN IN VS CODE

Click the account icon

Click on “backup and sync settings”

On the top, click on “sign in”

Select “Sign in with GitHub”

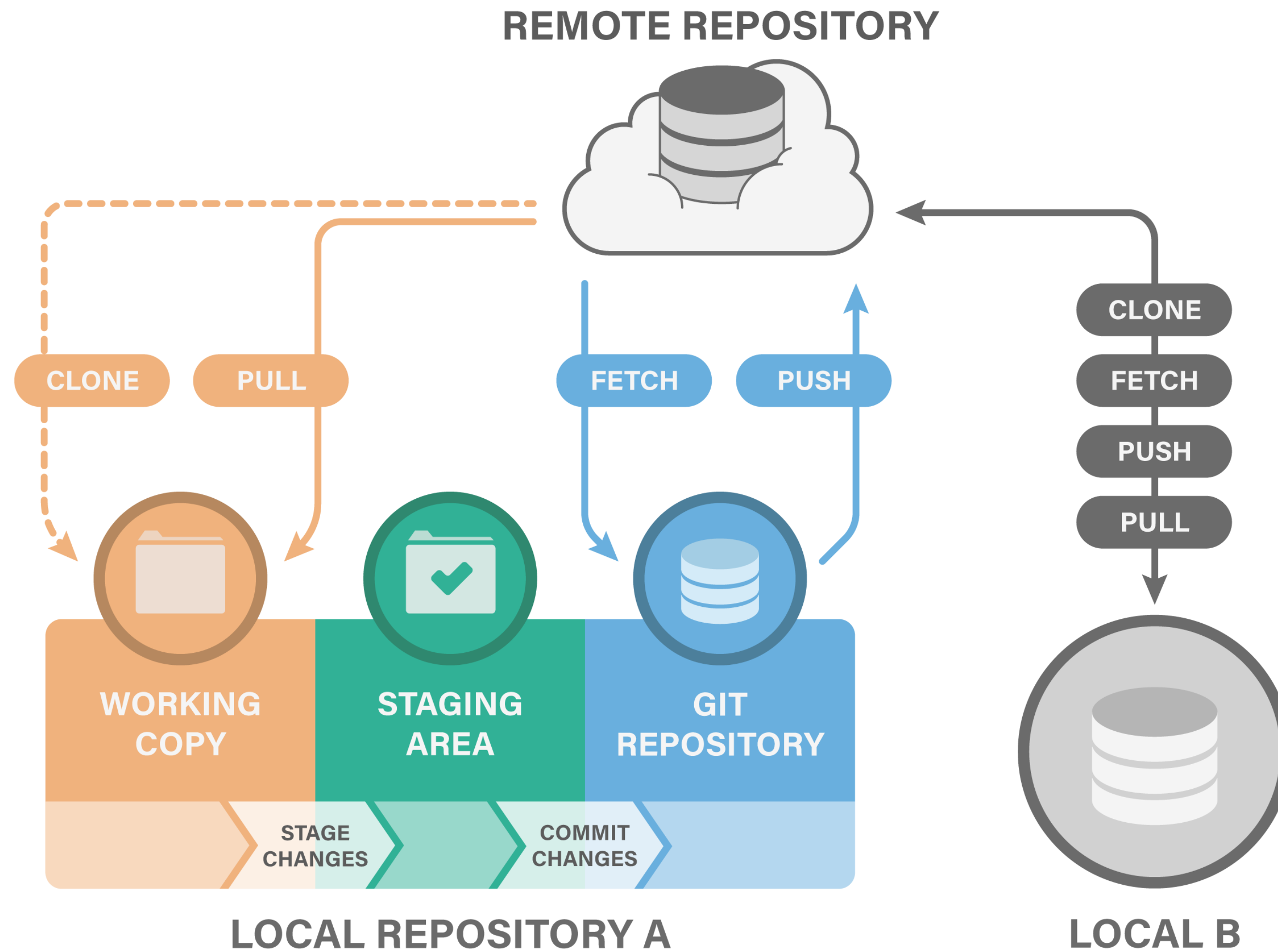
Follow the login instructions

3 CREATE A REMOTE REPO

Follow along!

4 ADD A REMOTE

What Are Remotes?



Cloning a Repo

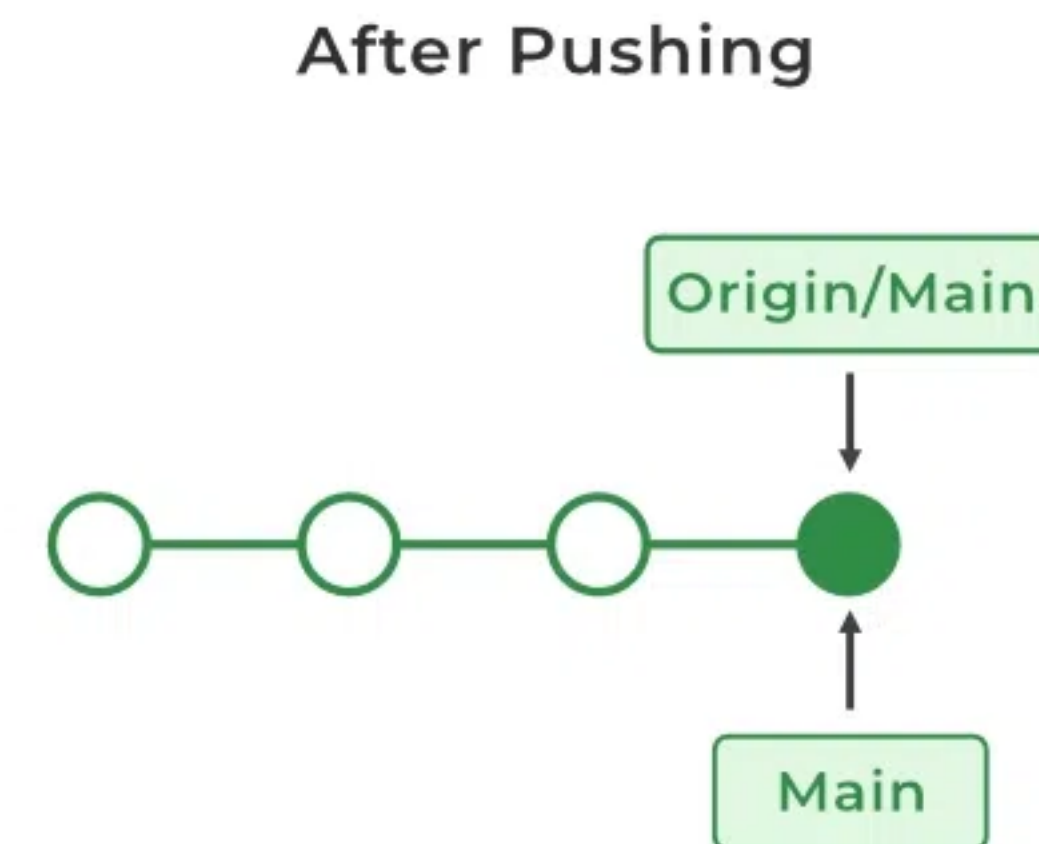
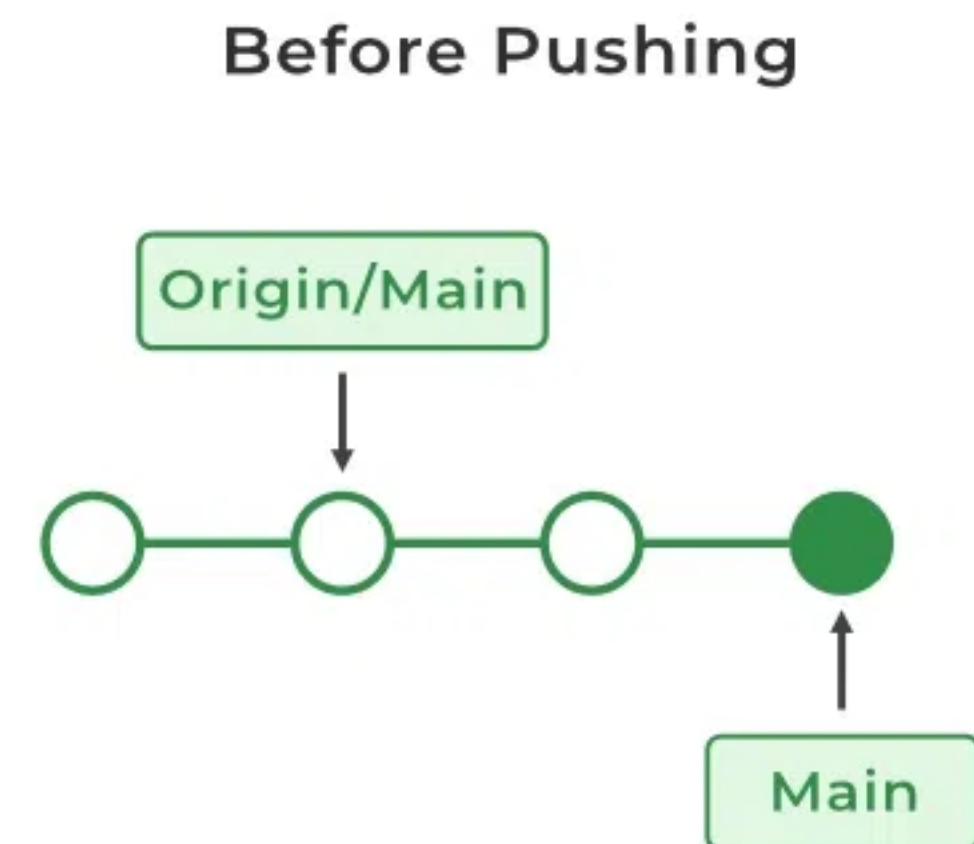
```
→ ~ git clone <url>
```

Checking Remote Connections

```
→ ~ git remote -v
```


Pushing Changes

```
→ ~ git push origin main
```



Pulling Changes

→ ~ `git pull origin main`

```
$ git remote -v
origin https://github.com/frankguitar11/Space_Shooter_Pro.git (fetch)
origin https://github.com/frankguitar11/Space_Shooter_Pro.git (push)

Frankie Four@DESKTOP-83D9DLR MINGW64 /f/Unity/Unity Repos/GameDevHQ Course/Space
Shooter Pro (master)
$ git pull origin master
fatal: couldn't find remote ref master

Frankie Four@DESKTOP-83D9DLR MINGW64 /f/Unity/Unity Repos/GameDevHQ Course/Space
Shooter Pro (master)
$ git pull origin main
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 1.09 KiB | 185.00 KiB/s, done.
From https://github.com/frankguitar11/Space_Shooter_Pro
 * branch          main          -> FETCH_HEAD
 * [new branch]    main          -> origin/main

Frankie Four@DESKTOP-83D9DLR MINGW64 /f/Unity/Unity Repos/GameDevHQ Course/Space
Shooter Pro (master)
$
```

Fetch vs Pull

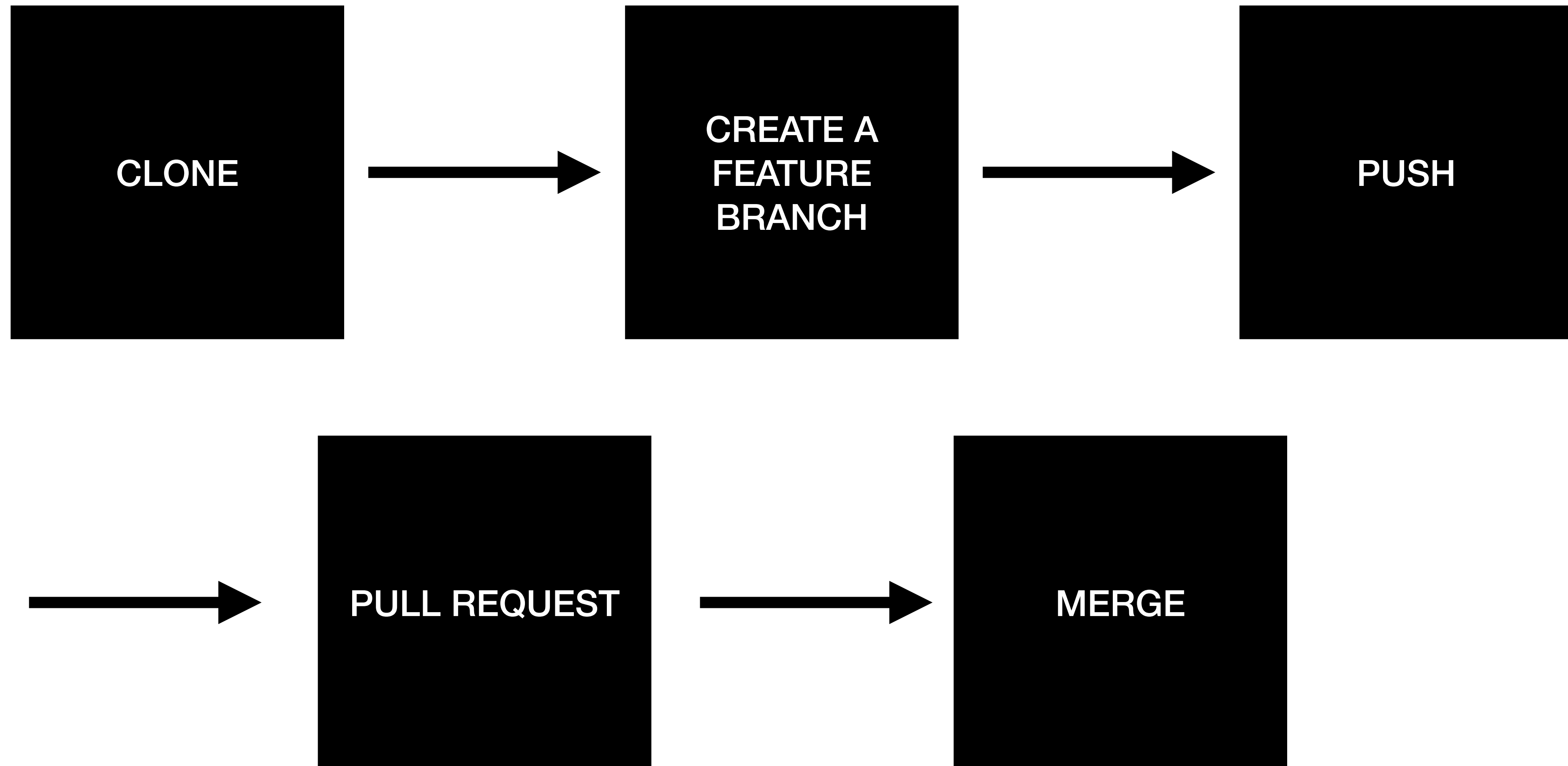
→ ~ `git fetch`

Git Pull	Git Fetch
Git pull is the combination of two commands git fetch and git merge.	Git fetch command will fetch the remote repository to the local machine.
It will fetch the changes and merges with the repository.	It will only fetch the branch without any merge operation.
A more efficient method for updating your local branch is to use git pull.	Git fetch is frequently used as a first step before using 'git merge' to combine updates.

Working with Branches Remotely



Collaboration Flow



Pull / Merge Requests

Showing 1 changed file with 3 additions and 2 deletions.

UnifiedSplit

5 README.md

<>📄⋮

... @@ -1,5 +1,6 @@

1- # A Practice Homework GitHub Repository

2

3This repository is a demo repository that you can use to practice your GitHub and git skills while working

4- thought he online lessons in the [introduction to earth data science textbook on earthdatascience.org] (https://www.earthdatascience.org/courses/intro-to-earth-data-science/git-github/version-control/). We will not merge any pull requests here but you can practice opening and closing them here if you want!

5

1+ # A Practice Homework GitHub Repository - Intro to Earth Data Science Textbook Demo

2

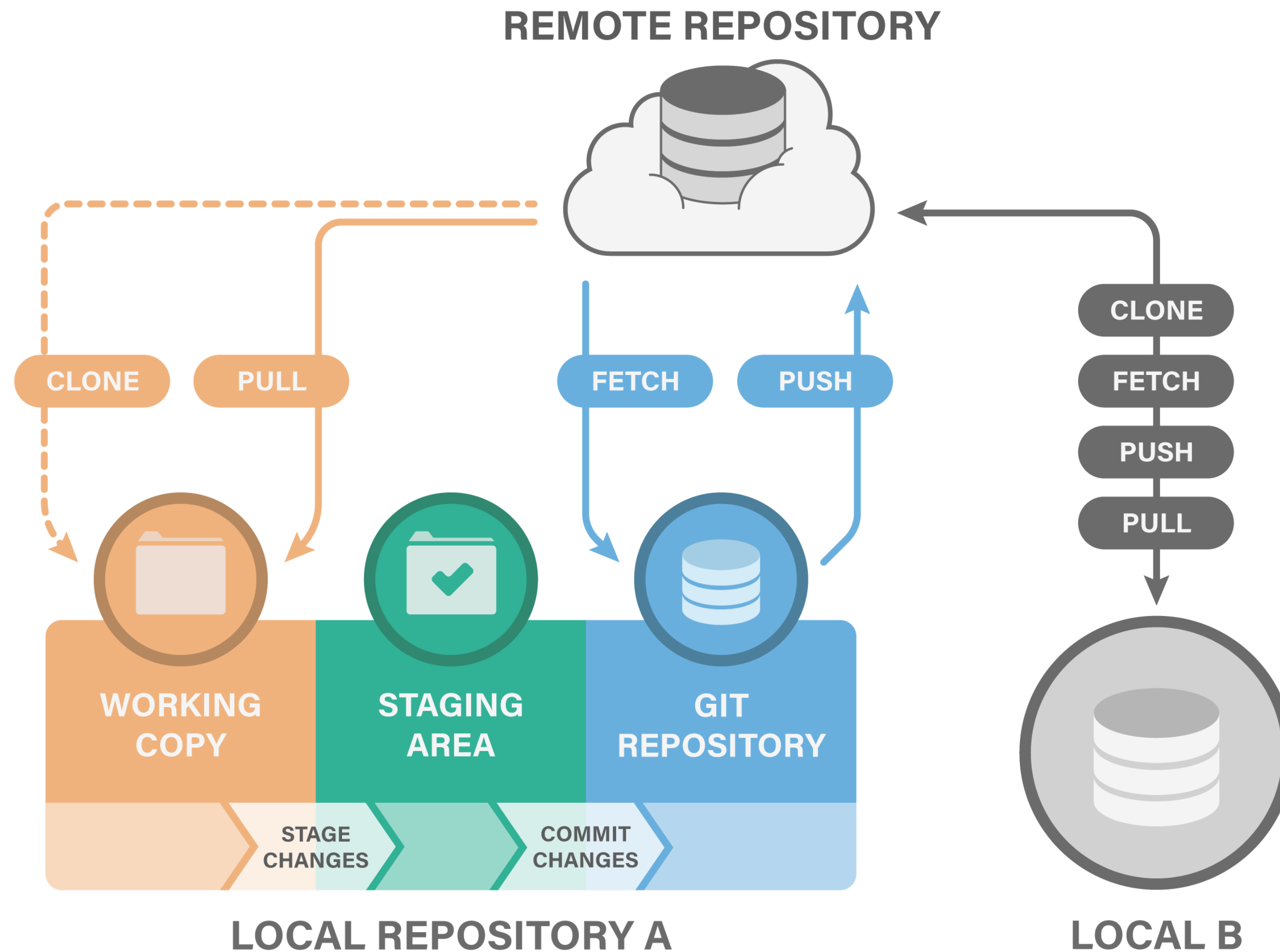
3This repository is a demo repository that you can use to practice your GitHub and git skills while working

4+ through the online lessons in the [introduction to earth data science textbook on earthdatascience.org] (https://www.earthdatascience.org/courses/intro-to-earth-data-science/git-github/version-control/). We will not merge any pull requests here but you can practice opening and closing them here if you want!

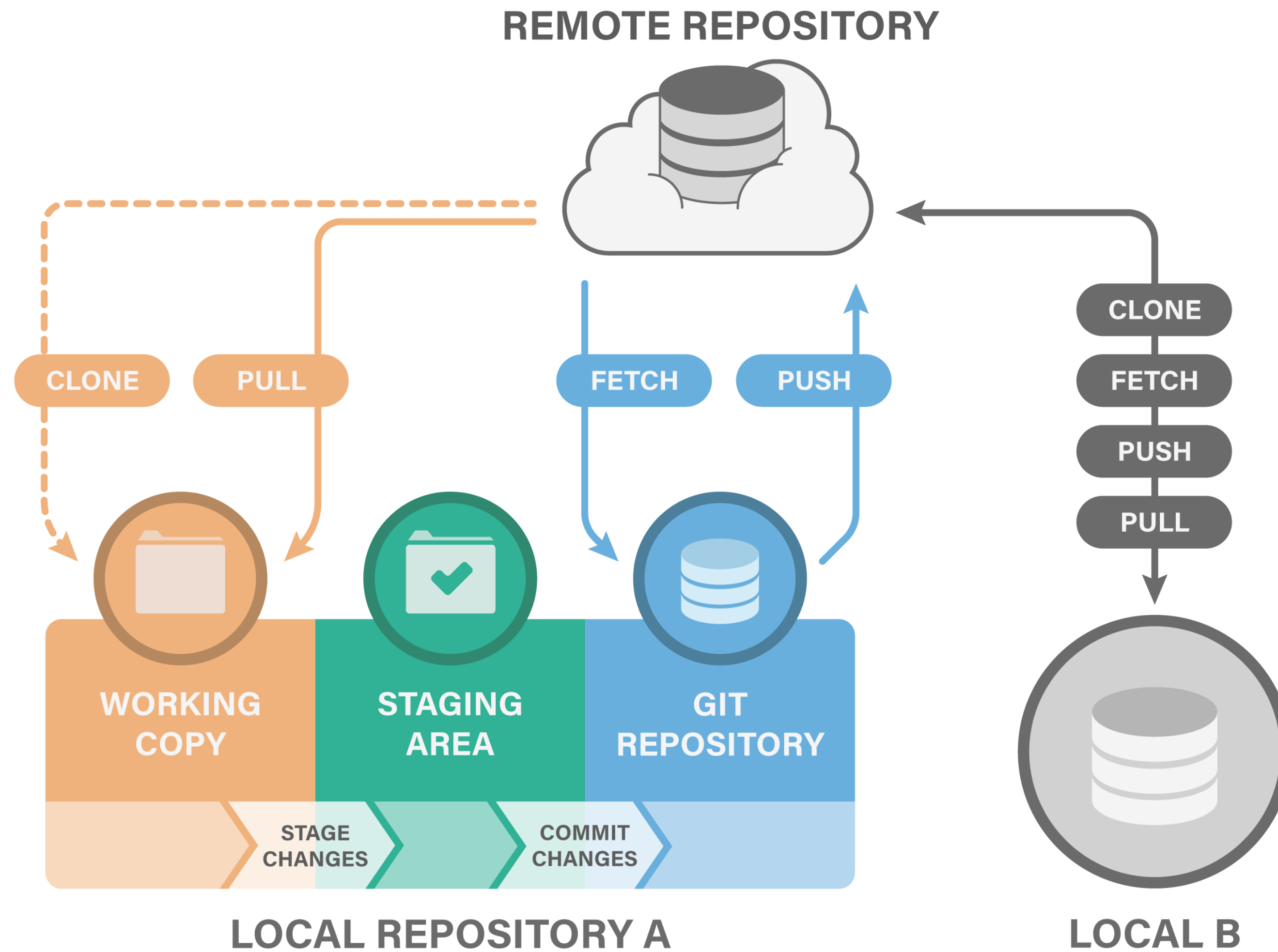
5

6+ Here is some more text.

What Are Remotes?



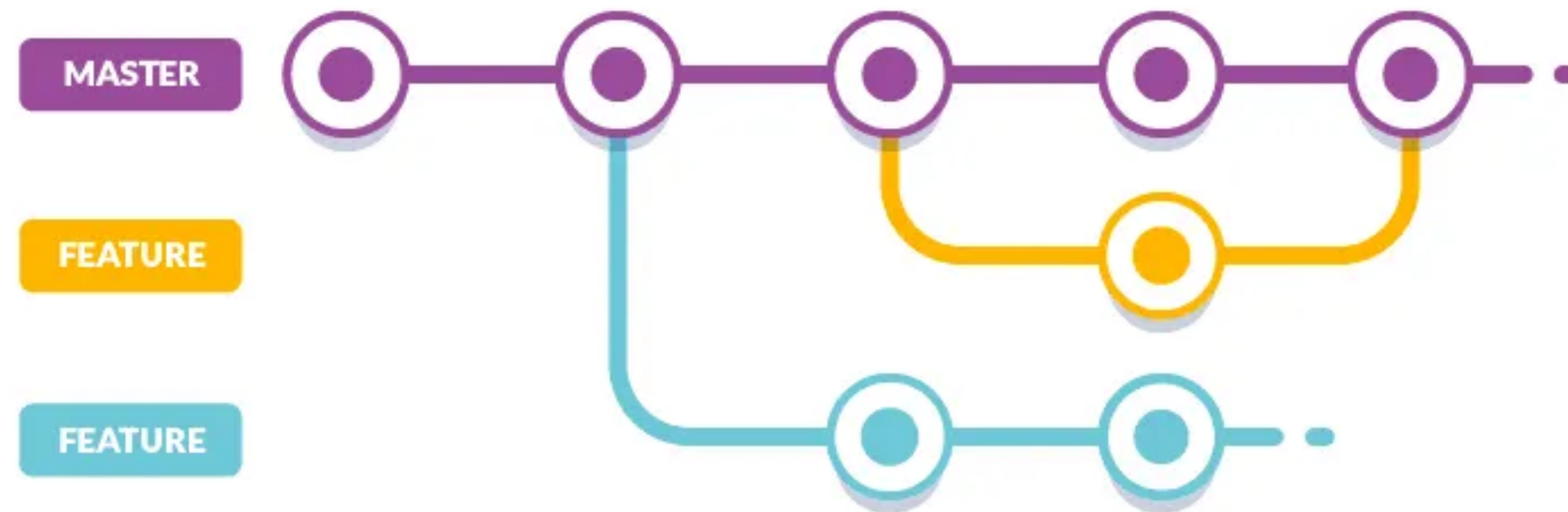
What Are Remotes?



Feature Branch Workflow

Description: Each new feature is developed in its own branch, separate from the main branch (main or master). This keeps the main branch free from unstable code, making it always deployable. Developers merge feature branches into main only after completion and testing.

Usage: Ideal for teams that want to keep a clean history in the main branch and for projects where continuous delivery is essential.

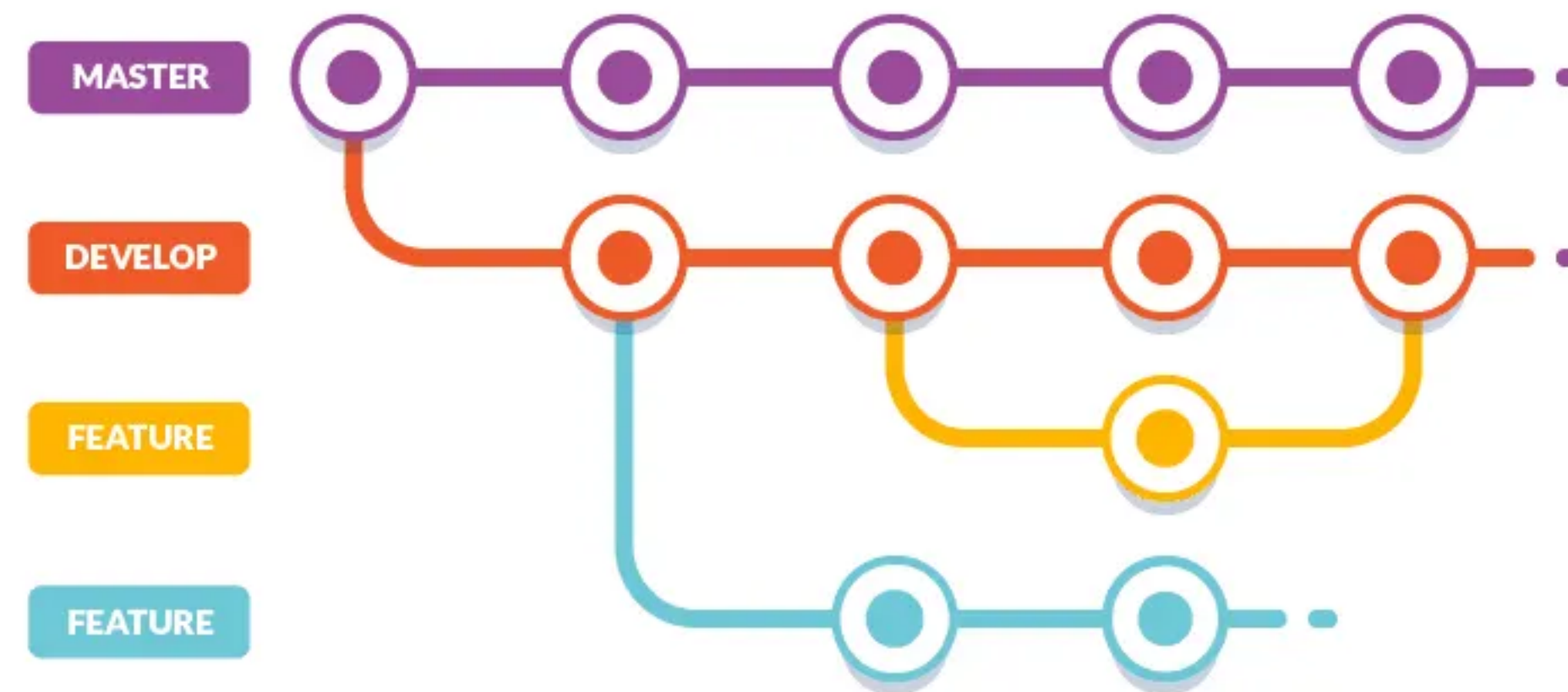


Gitflow Workflow

Description: A more structured extension of the feature branch workflow, Gitflow defines specific branch types for different purposes:

- Feature branches: For new features.
- Develop branch: Integration branch for features, aiming for the next release.
- Release branches: For preparing releases, allowing for minor bug fixes and preparing metadata for a release.
- Hotfix branches: Directly off main for quick fixes to be released outside of the regular release cycle.

Usage: Suitable for projects with scheduled releases and the need for parallel development (features, hotfixes).



Trunk-Based Development

Description: Developers work in short-lived branches or directly in the trunk (main or master). The goal is to avoid long-living branches and integrate changes frequently. This requires a robust automated testing and continuous integration setup to maintain stability.

Usage: Ideal for teams practicing continuous integration and delivery, aiming for rapid iterations and releases.



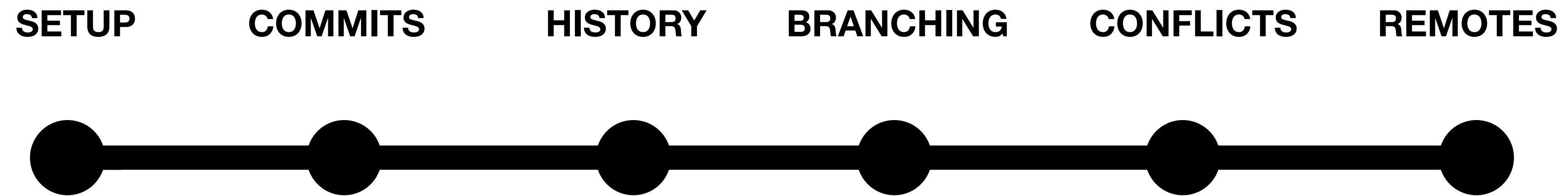
Forking Workflow

Description: Instead of using a single server-side repository, each developer pushes to their own forked repository. Contributions to the main project are made through pull requests from these personal repositories. This workflow is often used in open-source projects.

Usage: Best for projects with external contributors. It allows project maintainers to strictly control who can push changes directly to the main project.

Best Practices, Advanced Tips & Wrap-up

Workshop Recap (Visual Map)



Best Practices

- Always create Pull Requests (PRs): for visibility, review, and history.
- Write clear commit messages & PR descriptions: explain what and why, not just how.
- Resolve conflicts locally, not on GitHub UI: Better visibility and control.
- Agree on a team branching model: GitHub Flow, GitLab Flow, or Gitflow, don't mix styles.
- Keep branches small, merge often.

Commit Message Guidelines

- Capitalization and Punctuation: Capitalize the first word and do not end in punctuation. If using Conventional Commits, remember to use all lowercase.
- Mood: Use imperative mood in the subject line. Example – Add fix for dark mode toggle state. Imperative mood gives the tone you are giving an order or request.
- Type of Commit: Specify the type of commit. It is recommended and can be even more beneficial to have a consistent set of words to describe your changes. Example: Bugfix, Update, Refactor, Bump, and so on. Length: The first line should ideally be no longer than 50 characters, and the body should be restricted to 72 characters.
- Content: Be direct, try to eliminate filler words and phrases in these sentences (examples: though, maybe, I think, kind of). Think like a journalist.

Commit Message Guidelines

Good

- feat: improve performance with lazy load implementation for images
- chore: update npm dependency to latest version
- Fix bug preventing users from submitting the subscribe form
- Update incorrect client phone number within footer body per client request

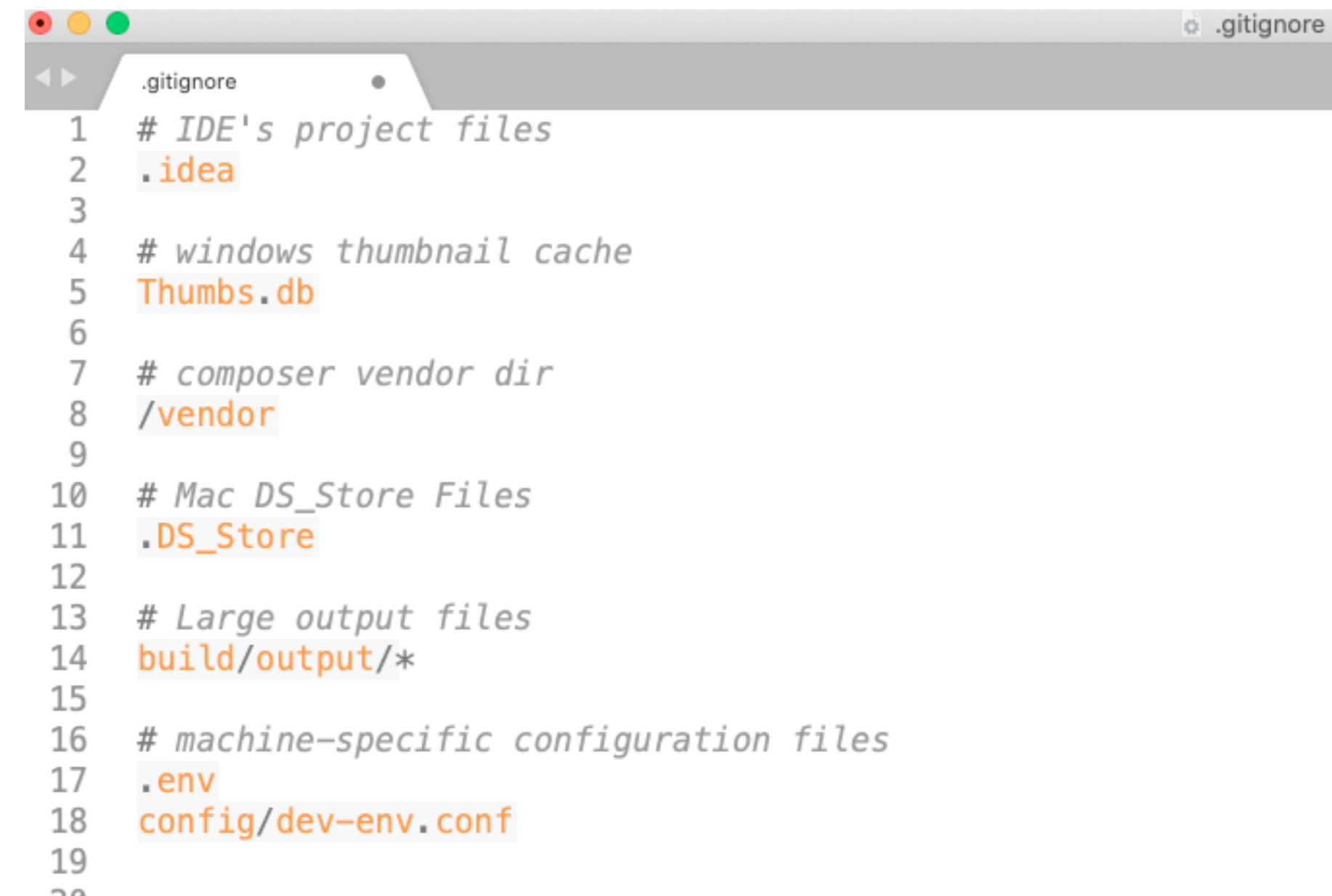
Bad

- fixed bug on landing page
- Changed style
- oops
- I think I fixed it this time?
- empty commit messages

Clean History Practices

Avoid committing generated files.

<https://github.com/github/gitignore>

A screenshot of a code editor window showing a .gitignore file. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a tab labeled '.gitignore'. The file content is as follows:

```
1 # IDE's project files
2 .idea
3
4 # windows thumbnail cache
5 Thumbs.db
6
7 # composer vendor dir
8 /vendor
9
10 # Mac DS_Store Files
11 .DS_Store
12
13 # Large output files
14 build/output/*
15
16 # machine-specific configuration files
17 .env
18 config/dev-env.conf
19
20 ~
```

Wrap-up & Q&A

- Feedback & reflections.

Extras - git stash

```
→ ~ git stash
```

WHAT IT DOES

Saves your local changes (working directory + staged) to a stack without committing. Cleans your working directory so you can switch branches.

USE CASES

Need to quickly switch branches without losing unfinished work. Experimenting or testing changes temporarily.

NOTES

Retrieve stashed changes with: git stash apply (keeps stash) or git stash pop (removes stash). Stashes are local only, not pushed to remote.

Extras - git cherry-pick

```
→ ~ git cherry-pick <commit-hash>
```

WHAT IT DOES

Applies a commit from another branch onto the current branch.

USE CASES

Take a bug fix or feature from a different branch without merging the whole branch.

NOTES

Can create merge conflicts; resolve like a normal conflict. Produces a new commit with a different hash.

Extras - git rebase

```
→ ~ git rebase main
```

WHAT IT DOES

Moves or replays commits from one branch onto another base commit.
Rewrites commit history linearly.

USE CASES

Keep feature branch up-to-date with main branch.
Clean up commit history before merging.

NOTES

Avoid rebasing public/shared branches—rewriting history can break others' clones.

Extras - git squash

```
→ ~ git rebase -i HEAD~3 # mark commits as 'squash'  
or 'fixup'
```

WHAT IT DOES

Merges multiple commits into a single commit.
Often used with interactive rebase (`git rebase -i`) before merging.

USE CASES

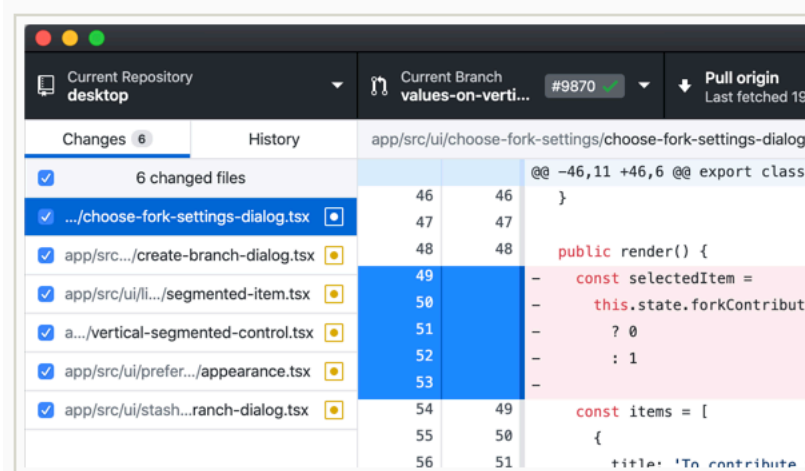
Clean up messy commit history.
Group related changes into a single commit for clarity.

NOTES

Squash changes commit hashes, so avoid on shared branches.
After squashing, you may need to force-push if branch is remote.

Explore Git GUIs/IDEs

<https://git-scm.com/tools/guis>

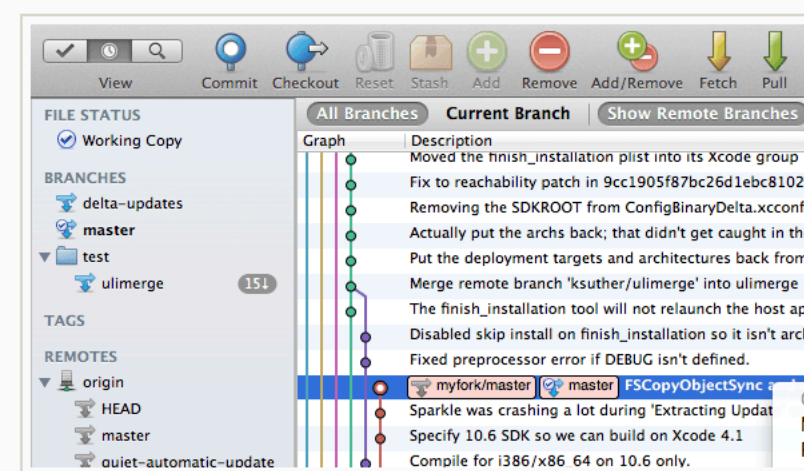


GitHub Desktop

Platforms: Mac, Windows

Price: Free

License: MIT

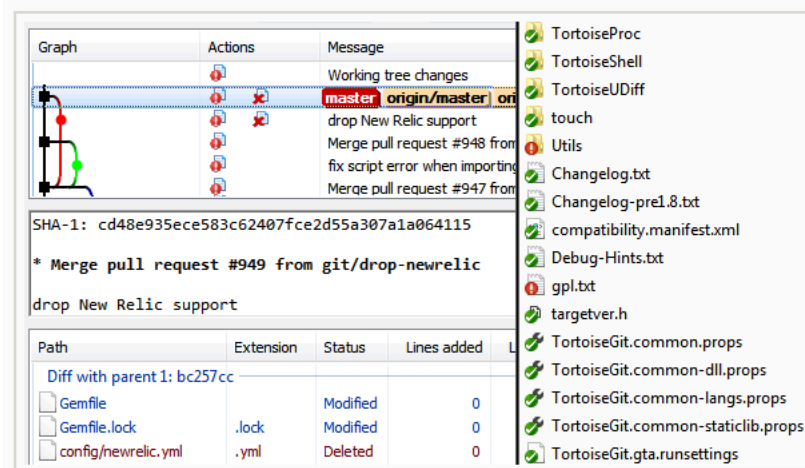


SourceTree

Platforms: Mac, Windows

Price: Free

License: Proprietary

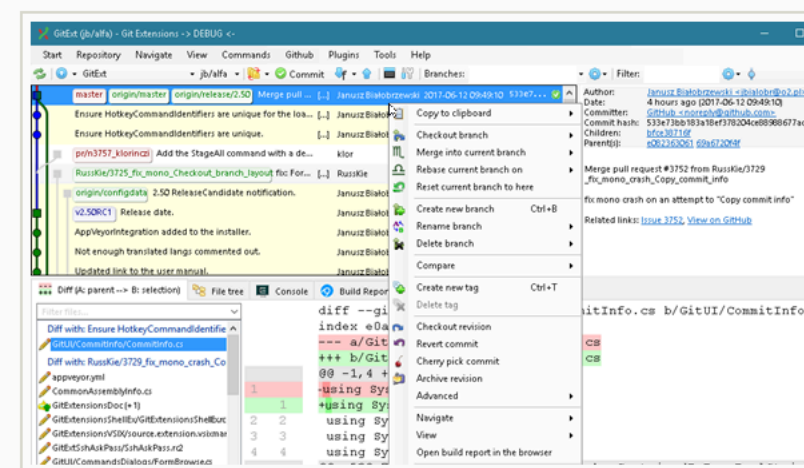


TortoiseGit

Platforms: Windows

Price: Free

License: GNU GPL



Git Extensions

Platforms: Windows

Price: Free

License: GNU GPL